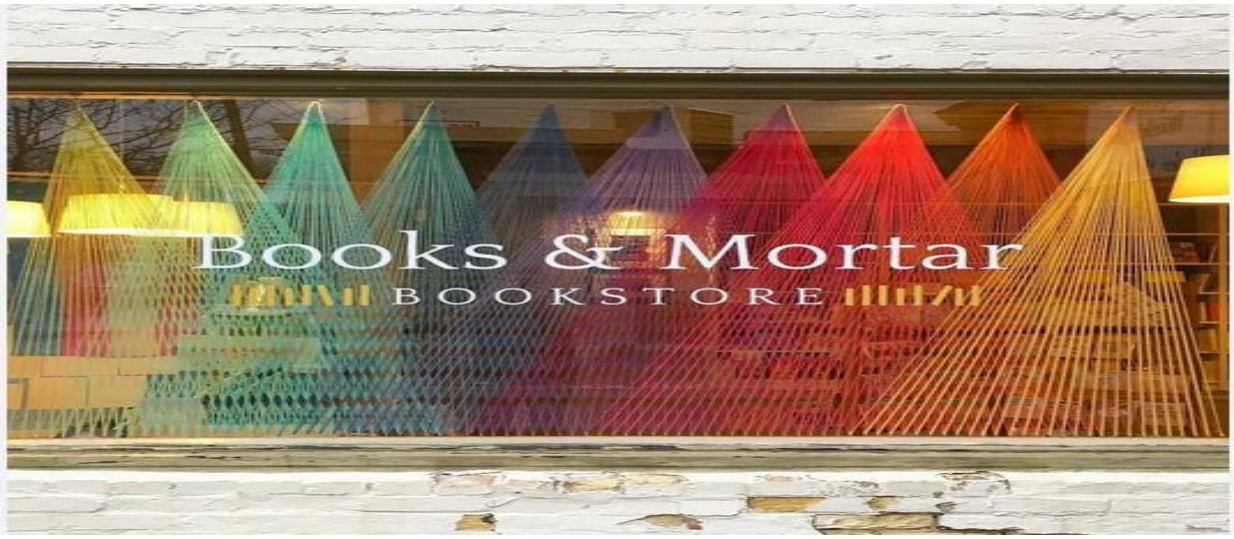
Data Mining Project

Caden Raleigh



I. Objective

The Books & Mortar Bookstore is looking to optimize their sale strategy and increase their revenue. Thier management has enlisted us to analyze their sales data and identify the patterns in their customers buying habits. The SPSS Modeler will be employed to help us take on and complete this task. Our goal is to data mine their sales data and help them cater their book bundles and promotions towards to the consumer base. We hope to boost Books & Mortars sales and help increase the efficiency of their marketing. Success will be measured using rule support, deployability, and lift. We believe these metrics will help us identify positive associations between book purchases as well as how easily our recommendations can be implemented into Books & Mortar's sales strategy.

II. Data Set Description

The data we are analyzing consists of the last 200 transactions from the Books & Mortar Bookstore. The book categories in the data are ChildBks, YouthBks, CookBks, DoItYBks, RefBks, ArtBks, GeogBks, ItalAtlsas, ItalArt, and Florence. For data preparation the transaction data was imported to the software where our team had to convert the data from continuous to flag variables. This was done because we are dealing with categories of books and not numeric data. Next, we set the role of the fields to both input and output as we want to see the association rules, which are the outputs, that can be created from the imputed data.

Field	Measurement	Values	Missing	Check	Role
ChildBks	Flag	1.0/0.0		None	Both
YouthBks	Flag	1.0/0.0		None	Both
CookBks	Flag	1.0/0.0		None	Both
DoItYBks	Flag	1.0/0.0		None	Both
RefBks	Flag	1.0/0.0		None	Both
ArtBks	Flag	1.0/0.0		None	Both
GeogBks	Flag	1.0/0.0		None	Both
ItalCook	Flag	1.0/0.0		None	Both
ItalAtlas	Flag	1.0/0.0		None	Both
ItalArt	Flag	1.0/0.0		None	Both
Florence	Flag	1.0/0.0		None	Both

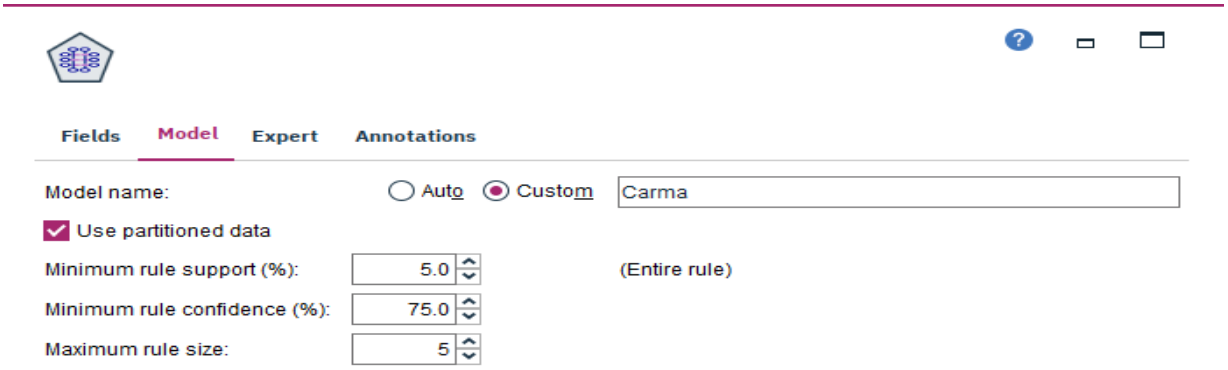
III. Data Mining Sessions

For our data mining tasks we employed the use of three different algorithms in the IBM SPSS Modeler which were the Carma, Apriori, and Association Rules nodes. This will allow us to analyze the associations found in all the algorithms and recommend rules show up more than once with a high rule support and deployability.

A. Carma

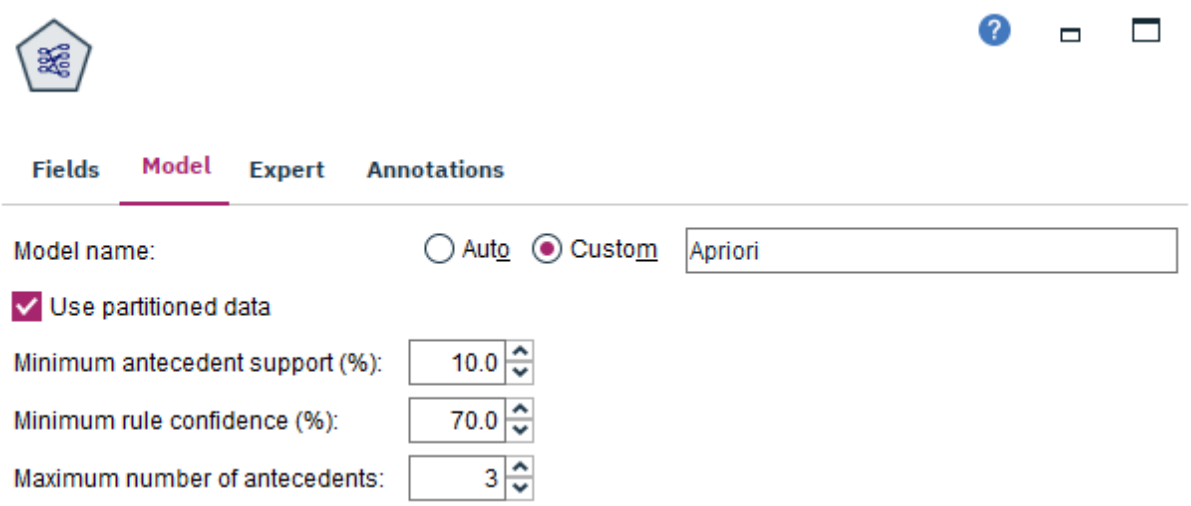
First, we used the Carma algorithm to data mine the transaction history. We set all variables as inputs and then set the parameters for the model. The minimum rule support was set to 5% to ensure that the associations found were meaningful and did not happen just a couple of times in the data. Next, the minimum rule confidence was set to 70% to make sure the model only

showed strong associations. Lastly, the maximum rule size was set to five, so the model didn't produce association rules so complex that they could not be realistically implemented.



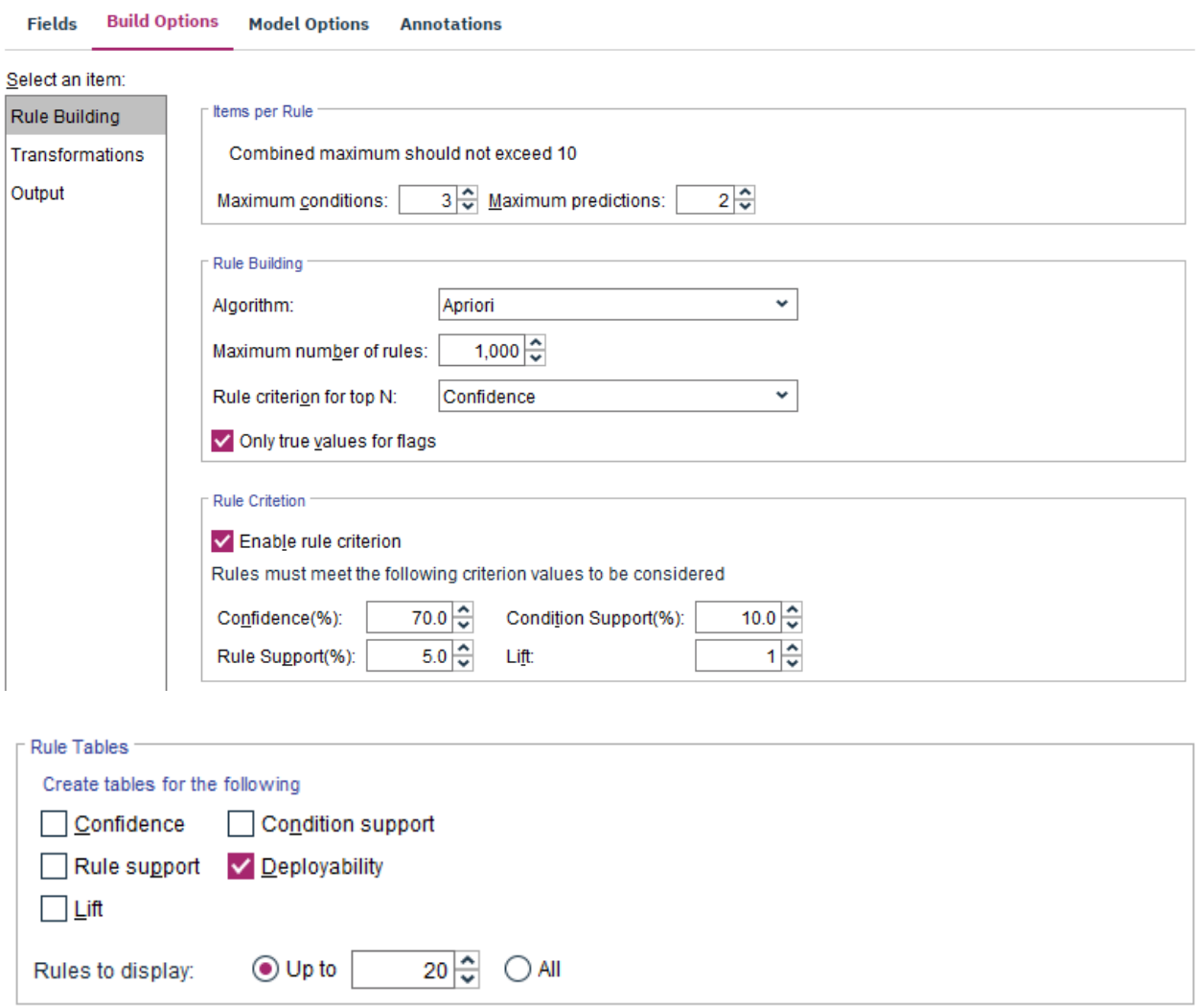
B. Apriori

The next algorithm we employed in the SPSS Modeler was Apriori. Similar to Carma we set all variables for the input, but for this algorithm they also had to be added to the output field. For the model parameters we set the antecedent support to 10% to ensure that the algorithm found patterns that showed up frequently and were not rare events. Again, we set the rule confidence to 70% to keep the algorithm's findings limited to strong correlations. Then the max number of antecedents was limited to three to make the association rules easier to act upon.



C. Association Rules

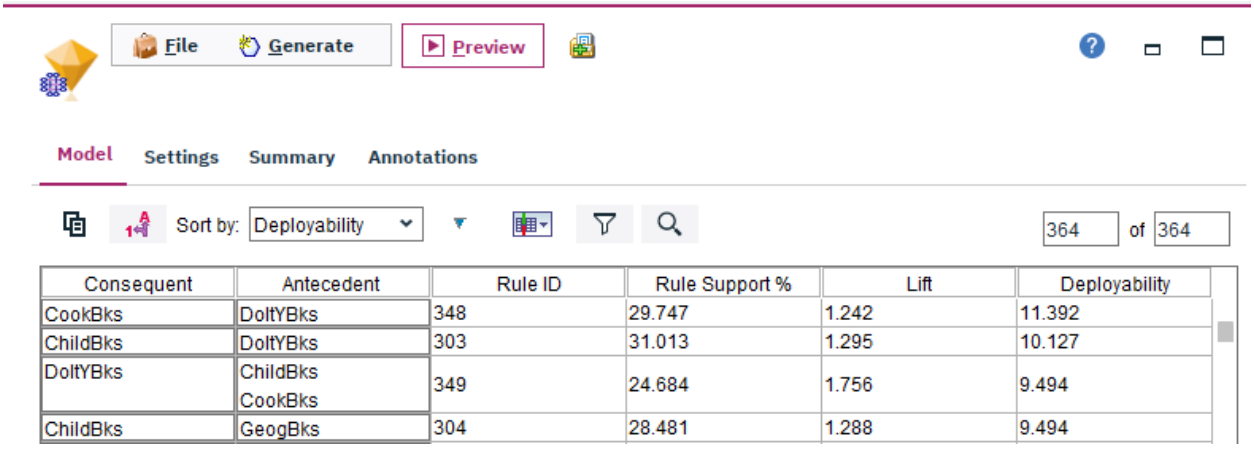
The final algorithm that was used was the association rules algorithm. To keep the parameter consistent with the other algorithms variables were set to both input and output. The maximum number of conditions was set to three and the maximum predictions was set to two and we limited the algorithm to making at max of 1000 rules. The rule criterion was then set to a confidence of 70%, a rule support of 5%, a condition support of 10%, and a lift of one. Then we set the rule table to create rules for deployability.



IV. Findings

A. Carma

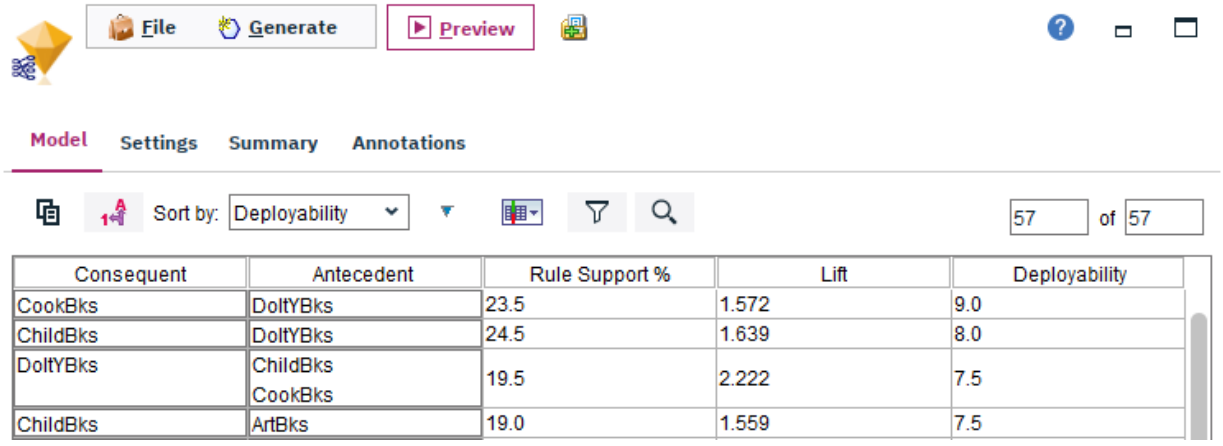
For the Carma algorithm our top four associations based on the metrics we chose were If CookBks, Then DoItYBBks, If ChildBks, Then DoItYBBks, If DoItYBBks, then ChildBks and CookBks, and lastly If ChildBks, Then GeogBks.



Consequent	Antecedent	Rule ID	Rule Support %	Lift	Deployability
CookBks	DoItYBks	348	29.747	1.242	11.392
ChildBks	DoItYBks	303	31.013	1.295	10.127
DoItYBks	ChildBks CookBks	349	24.684	1.756	9.494
ChildBks	GeogBks	304	28.481	1.288	9.494

B. Apriori

For the Apriori algorithms our findings were similar to Carma with the top four associations being If CookBks, Then DoItYBBks, If ChildBks, Then DoItYBBks, If DoItYBBks, then ChildBks and CookBks, and lastly If ChildBks, Then ArtBks.



Consequent	Antecedent	Rule Support %	Lift	Deployability
CookBks	DoItYBks	23.5	1.572	9.0
ChildBks	DoItYBks	24.5	1.639	8.0
DoItYBks	ChildBks CookBks	19.5	2.222	7.5
ChildBks	ArtBks	19.0	1.559	7.5

C. Association Rules

For our final alorithm the top rules were If DoItYBks, Then CookBks, If DoItYBks, Then ChildBks, If GeogBks, Then ChildBks, and If ArtBks, Then ChildBks.

Rules Table

Most Interesting Rules by Deployability

					Other Evaluation Statistics			
Rank	Rule ID	Condition	Prediction	Sorted By Deployability(%)	Condition Support (%)	Confidence (%)	Rule Support (%)	Lift
1	77			11.39	41.14	72.31	29.75	1.24
2	62			10.13	41.14	75.38	31.01	1.29
3	63			9.49	37.97	75.00	28.48	1.29
4	82			9.49	33.54	71.70	24.05	1.23

Most Interesting Rules by Deployability

Rule 77- If DoItYBks **Then** CookBks
Rule 62- If DoItYBks **Then** ChildBks
Rule 63- If GeogBks **Then** ChildBks
Rule 82- If ArtBks **Then** ChildBks

V. Recommendations

Based on the finding of the three algorithms used we recommend that Books & Mortar cater their book bundles with DoItYBks and CookBks and ChildBks. As well as ChildBks with ArtBks and GeogBks. These recommendations are based on the rules with the most deployability so they should be the easiest to implement within Books & Mortar marketing plans. With these recommendations we believe that Books & Mortar will have more successful sales and waste less resources on books that don't get bought together often. We would also recommend that these different book genres be placed in proximity, so the customer is more likely to see each while they are shopping.

(Unsupervised Clustering)

1. Objective

The primary objective of this project is to analyze the Iris dataset using unsupervised clustering techniques. As a data mining analyst, I was assigned this project by a client seeking to classify different species of iris flowers without prior labeled data. The dataset consists of four numerical features—sepal length, sepal width, petal length, and petal width—used to differentiate between three species of iris flowers: Setosa, Versicolor, and Virginica.

The main goals of this project are:

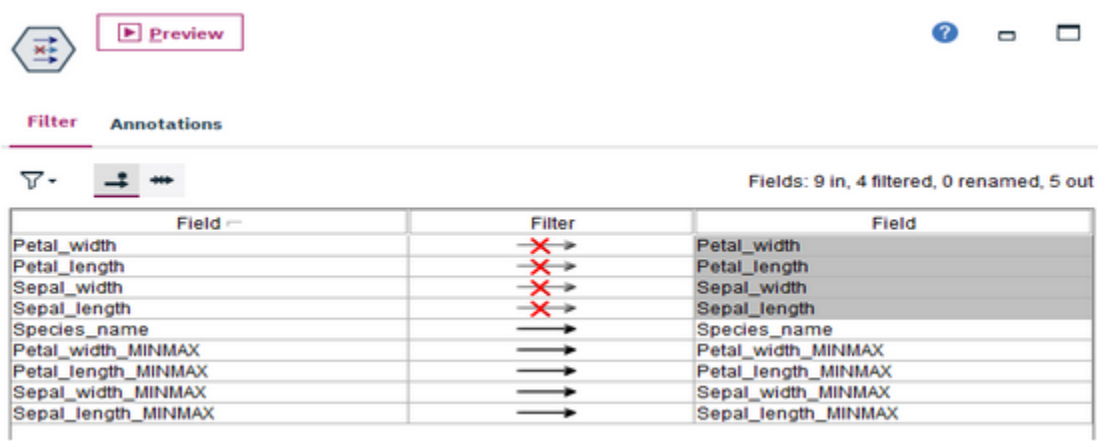
- To determine the best number of clusters for the dataset using various clustering algorithms.
- To compare clustering outcomes using two and three clusters to evaluate which better represents the data.
- To assess clustering performance using IBM SPSS Modeler and algorithms such as Two-Step clustering, Kohonen networks, K-Means, and Two-Step-AS.
- To normalize and preprocess the data to ensure optimal clustering performance.
- To measure the success of clustering based on internal validation metrics such as silhouette score, cluster cohesion, and separation.

2. Data Set Description

The dataset consists of the following attributes:

- Species_No: Numerical representation of the flower species (1 for Setosa, 2 for Versicolor, 3 for Virginica).
- Species_Name: Name of the Iris species.
- Petal_Width: Width of the petal (in mm).
- Petal_Length: Length of the petal (in mm).
- Sepal_Width: Width of the sepal (in mm).
- Sepal_Length: Length of the sepal (in mm).

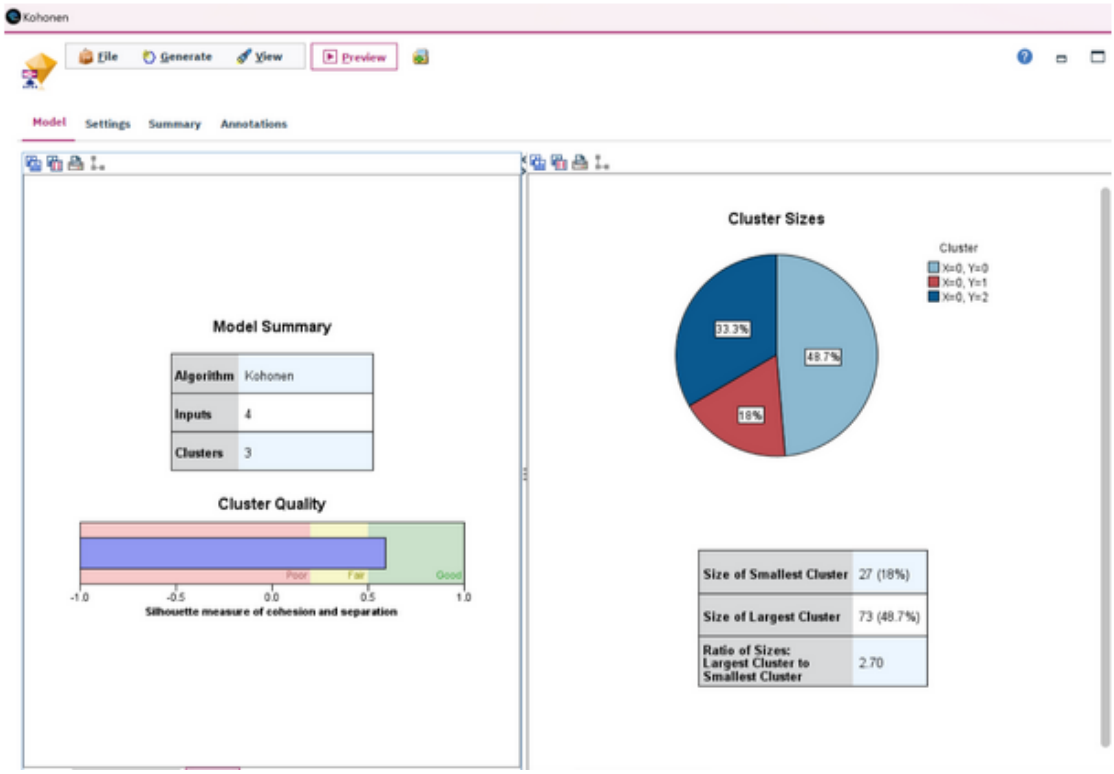
The dataset contains measurements for 150 instances, evenly distributed across three species. Before analysis, the data was prepared by normalizing measurements and ensuring consistency in format. We had to ensure that our data was normalized by using min max from (Set Global) and then by using the (Derive Node) we were able convert the data from [0,1], and then read it in using the type node. Afterwards we used a filter node to stop the first four.

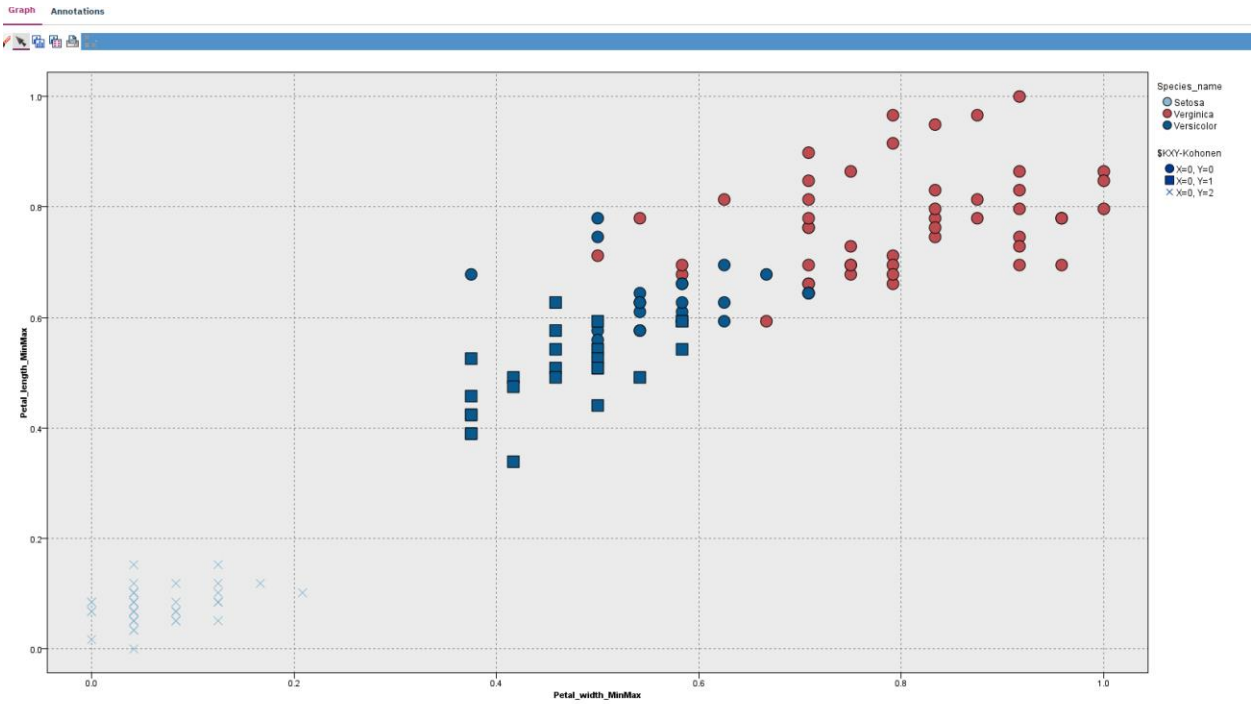


3. Data Mining Sessions

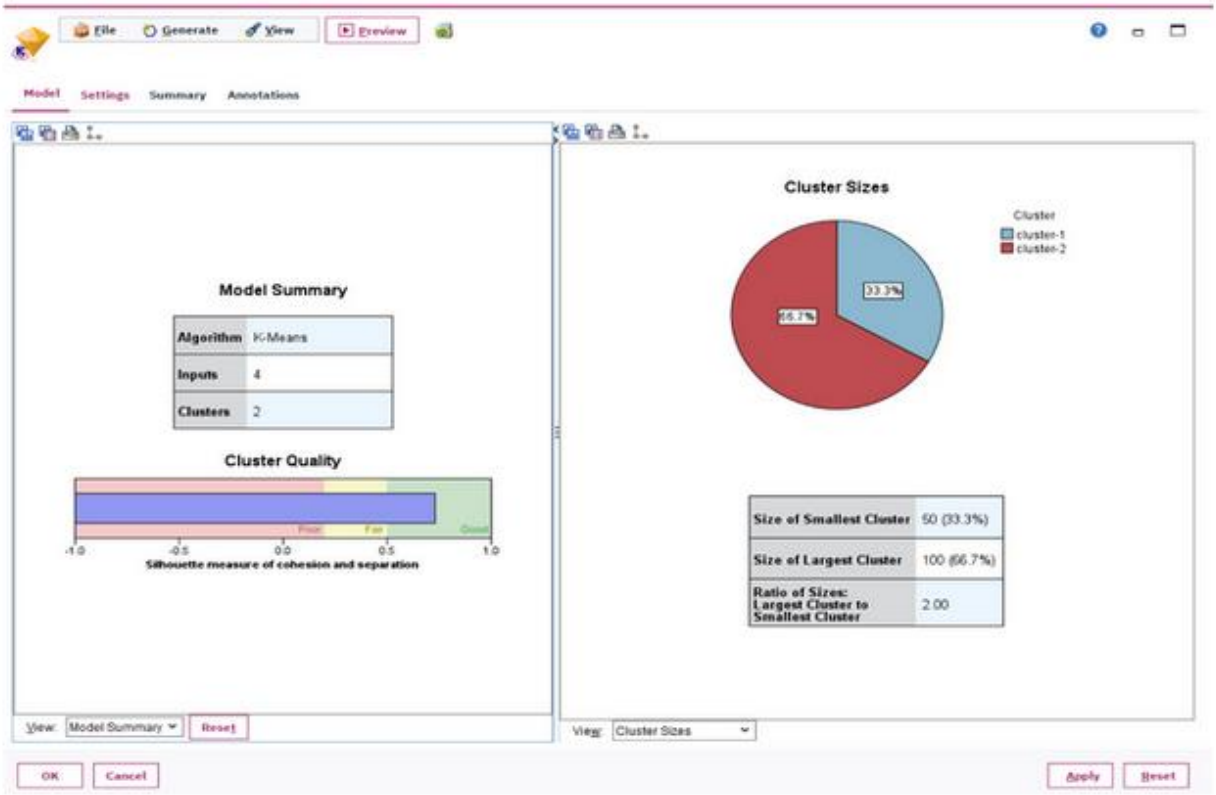
By applying statistical methods such as clustering, classification algorithms, or principal component analysis (PCA), we can further explore relationships among these attributes and determine distinguishing factors between the species. To achieve this, we tested different classification algorithms, including TwoStep, Kohonen, TwoStep-As, and K-Means, to compare their effectiveness in grouping and distinguishing species based on the dataset attributes.

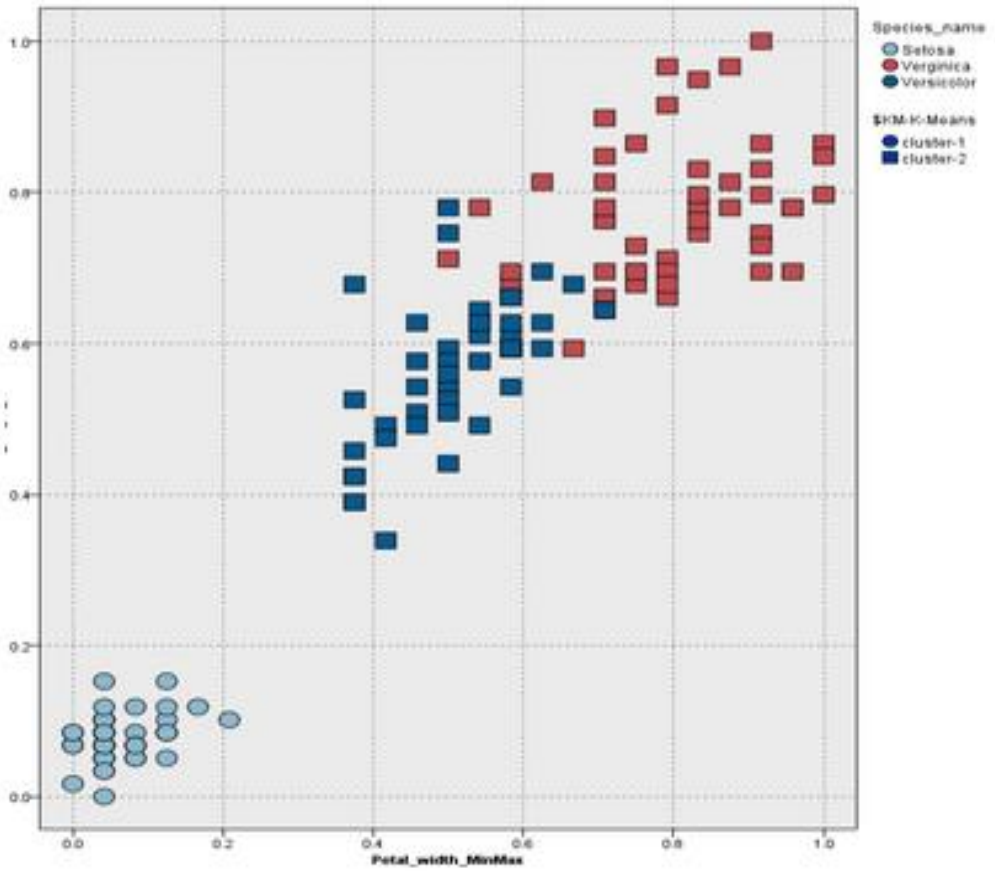
A. Kohonen - The Kohonen algorithm, a type of self-organizing map (SOM), was applied to the dataset using a 3x3 grid, resulting in multiple clusters. The model was trained with four input variables, which played a key role in determining the structure of the clusters. The relatively low quantization error suggests effective clustering and meaningful separation of data points. Predictor importance analysis highlights the most influential variables that guided the formation of clusters. By mapping high-dimensional data onto a lower-dimensional space while preserving topological relationships, the algorithm successfully identifies patterns and structures within the dataset. When using a graph to check for error there were very little errors.





B. The K-Means algorithm was applied to the dataset using four input variables, resulting in two distinct clusters. The model's silhouette measure indicates fair clustering quality, meaning the separation between clusters is moderate. The largest cluster contains 100 data points (66.7%), while the smallest cluster has 50 data points (33.3%), with a size ratio of 2:1. The most influential variable in cluster formation is pedal length, with its minimum and maximum values being the strongest differentiators between groups. This highlights the importance of pedal length in defining distinct patterns within the dataset. When looking at the graph there are a couple mistakes where the Verginica are supposed to be red boxs or blue instead of being mixed in with eachother.


File
Generate
View
Preview
Model
Settings
Summary
Annotations
Model Summary
Algorithm K-Means
Inputs 4
Clusters 2
Cluster Quality
Poor
Fair
Good
-1.0
-0.5
0.0
0.5
1.0
Silhouette measure of cohesion and separation
View: Model Summary
Reset
Cluster Sizes
Cluster
cluster-1
cluster-2
33.3%
66.7%
Size of Smallest Cluster 50 (33.3%)
Size of Largest Cluster 100 (66.7%)
Ratio of Sizes: Largest Cluster to Smallest Cluster 2.00
View: Cluster Sizes
OK
Cancel
Apply
Reset


Species_name
Setosa
Verginica
Versicolor
$KM-K-Means
cluster-1
cluster-2
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Petal_width_MinMax

C. The TwoStep-As algorithm was implemented with specific build options to optimize clustering for the given dataset. For the build options, a minimum of 2 and a maximum of 7 clusters were used to ensure a balance between too few or too many clusters. A feature tree was created using the Euclidean distance formula to compute similarities between data points, ensuring that the data was not standardized to preserve the original scaling. Notably, the sepal_width_minMax and sepal_length_minMax were excluded from the analysis to avoid distortions in clustering. In Cluster 2, the model identified two clusters rather than one, as Cluster 1 contained 50 records, while Cluster 2 contained 100 records. This resulted in a more refined clustering solution, improving the model's overall quality by appropriately splitting the data into clusters that reflected the underlying patterns.

TwoStep-AS

Model Information

Model Specifications

Minimum Number of Regular Clusters		2
Maximum Number of Regular Clusters		7
Adaptive Feature Selection		On
Feature Importance Method		Information Criterion
Information Criterion		Bayesian Information Criterion (BIC)
Distance Measure		Euclidean
Final Model	Number of Regular Clusters	2
	Number of Outlier Clusters	0
	Continuous Inputs	Petal_length_MinMax
		Petal_width_MinMax

Records Summary

Records	Number	Percent
Included	150	100.00
Excluded	0	0.00
Total	150	100.00

Excluded Inputs

Sepal_width_MinMax	Failed adaptive feature selection criteria
Sepal_length_MinMax	Failed adaptive feature selection criteria

Evaluation

Model Quality

Cluster	Number of Records	Goodness	Importance
Cluster-1	50	0.94	1.00
Cluster-2	100	0.61	1.00

Interpretation

Within Cluster Feature Importance

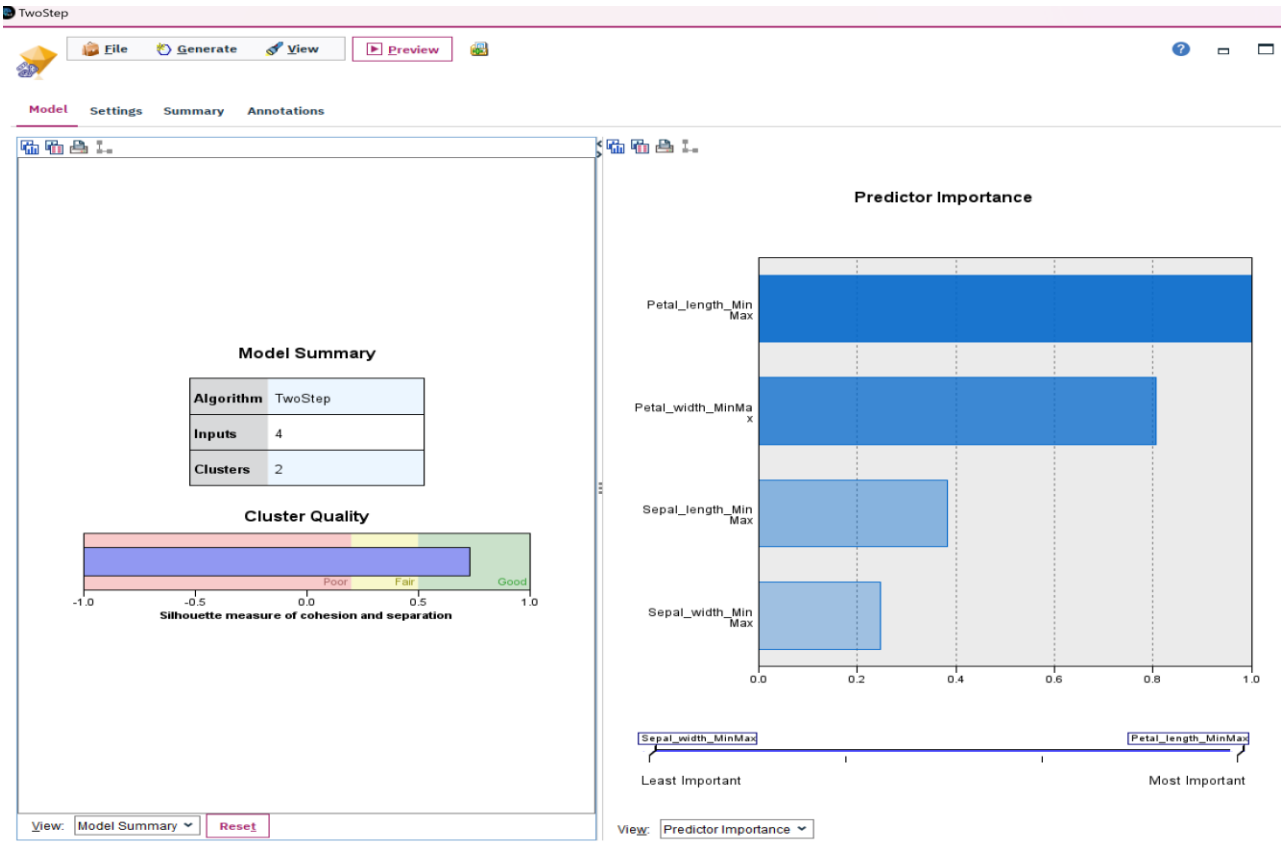
Cluster-1 Profile

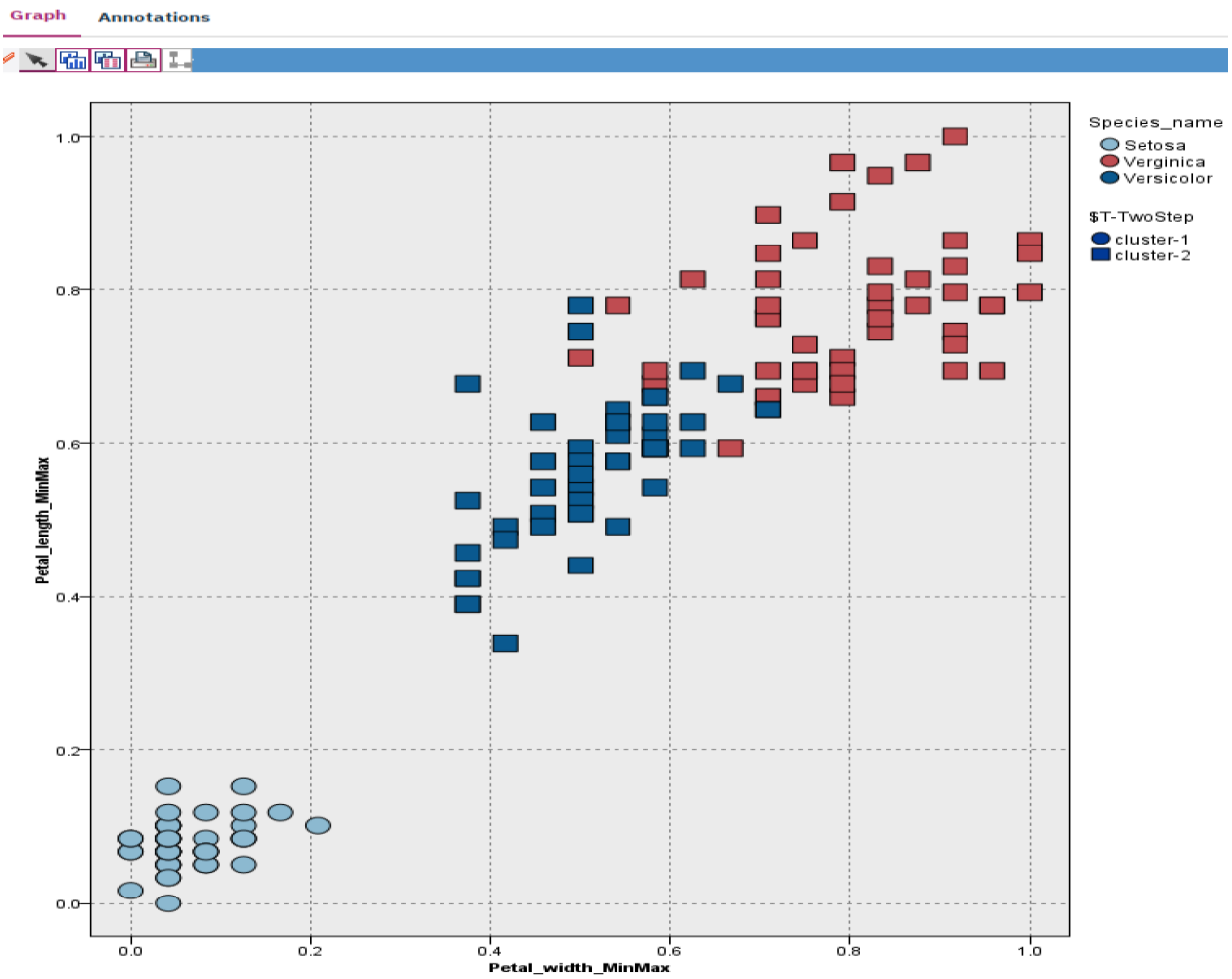
Input	Center	Within-Cluster Feature Importance
Petal_length_MinMax	0.08	1.00
Petal_width_MinMax	0.06	0.97

Cluster-2 Profile

Input	Center	Within-Cluster Feature Importance
Petal_length_MinMax	0.67	1.00
Petal_width_MinMax	0.65	0.97

D. The TwoStep algorithm was applied with four inputs, a minimum of 2 clusters and a maximum of 7 clusters, and utilized the Euclidean distance formula to measure similarities between data points. The clustering resulted in two clusters with a quality score of 0.7, which is considered satisfactory. The smallest cluster contained 50 records, representing 33.3% of the total data, while the larger cluster had 100 records, making up 66.7%. The petal_length_minMax feature emerged as the primary predictor, significantly influencing the separation between the clusters. The algorithm effectively partitioned the dataset, capturing the underlying patterns with clear distinctions in cluster sizes and quality. When running the graph there a few mistakes as they are not completely clustered separately but overall, the data came out pretty good.


TwoStep
File
Generate
View
Preview
Model
Settings
Summary
Annotations
Model Summary
Algorithm
TwoStep
Inputs
4
Clusters
2
Cluster Quality
Poor
Fair
Good
-1.0
-0.5
0.0
0.5
1.0
Silhouette measure of cohesion and separation
View:
Model Summary
Reset
Predictor Importance
Petal_length_Min Max
Petal_width_MinMa x
Sepal_length_Min Max
Sepal_width_Min Max
0.0
0.2
0.4
0.6
0.8
1.0
Sepal_width_MinMax
Petal_length_MinMax
Least Important
Most Important
View:
Predictor Importance



Recommendation: Based on the analysis, the TwoStep-As algorithm is the best option for the client due to its refined clustering solution, clear separation between clusters, and minimal distortion. It provides good cluster quality while excluding non-essential features, leading to more reliable results. The Kohonen algorithm also performed well but may require additional interpretation for accurate predictions, making it secondary to the TwoStep-As. The TwoStep algorithm is effective but has some minor clustering errors, and the K-Means algorithm was not ideal due to its clustering issues and errors in classification. Therefore, I recommend the client proceed with the TwoStep-As algorithm as their primary clustering method, with the potential to explore the Kohonen algorithm for supplementary insights.

(Supervised Learning)

I. Objective

The US Census Service would like our help in predicting the future median value of owner-occupied homes. They want us to use data they have collected to create a model so they may predict the trends in the housing market. The SPSS Modeler will be used to help take on and complete this task. To achieve our objective, we will use the data they have provided us with to train a classification model that will be used to predict the median value of owner-occupied homes. Success will be measured with the help of the mean absolute error, mean squared error, and lift. These metrics should help us predict what housing prices will be higher and what housing prices will be lower.

II. Data Set Description

The dataset that we have been provided by the Metropolitan Area Planning Council contains 506 cases. It provides data on 14 attributes which are listed below:

- **CRIM:** Per capita crime rates by town
- **ZN:** Proportion of residential land zone for lots over 25,000 sq.ft
- **INDUS:** Proportion of non-retail business acres per town
- **CHAS:** Charles River dummy variables
- **NOX:** Nitric oxides concentration in parts per 10 million
- **RM:** average number of rooms per dwelling
- **AGE:** Average number of rooms per dwelling (RM), proportion of owner-occupied units built prior to 1940
- **DIS:** Weighted distances to five Boston employment centres
- **RAD:** Index of accessibility to radial highways

- **TAX:** Full-value property-tax rate per $10,000
- **PTRATIO:** Pupil-teacher ratio by town
- **B:** 1000(Bk – 0.63)^2 where Bk is the proportion of blacks by town
- **LSTAT:** % lower status of the population
- **MEDV:** Median value of owner-occupied homes in $10K
- **CAT. MEDV:** Derived from the MEDV attribute, anything above $300,000 dollars is considered a 1, while anything below is considered a 0.

To prepare the data, the dataset was imported into the IBM SPSS Modeler where the MEDV attribute was filtered out and the CAT.MEDV attribute was set to be the output. The CHAS and CAT.MEDV attributes were set to be flags since they are Boolean, while the remaining attributes were set to continuous measurements.

Data Filter Types Annotations

Read Values Clear Values Clear All Values

Field	Measurement	Values	Missing	Check	Role
CRIM	Continuous	[0.01311,88...		None	Input
ZN	Continuous	[0.0,100.0]		None	Input
INDUS	Continuous	[0.46,27.74]		None	Input
CHAS	Flag	1.0/0.0		None	Input
NOX	Continuous	[0.394,0.871]		None	Input
RM	Continuous	[4.368,8.725]		None	Input
AGE	Continuous	[6.6,100.0]		None	Input
DIS	Continuous	[1.1691,10....		None	Input
RAD	Continuous	[1.0,24.0]		None	Input
TAX	Continuous	[188.0,711.0]		None	Input
PTRATIO	Continuous	[12.6,21.2]		None	Input
B	Continuous	[0.32,396.9]		None	Input
LSTAT	Continuous	[1.73,34.41]		None	Input
CAT. MEDV	Flag	1.0/0.0		None	Target

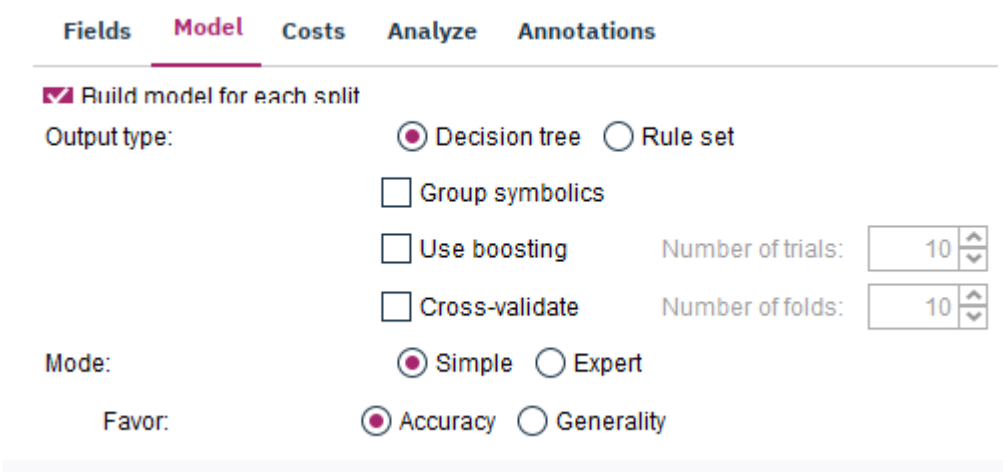
View current fields View unused field settings

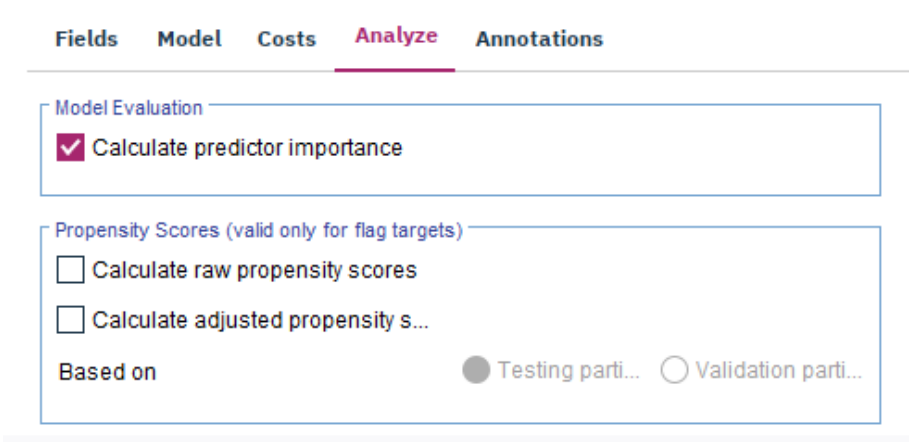
III. Data Mining Sessions

For our data mining sessions, we used seven different algorithms in the IBM SPSS Modeler which were neural network, C5.0, C&R tree, CHAID, QUEST, Tree-AS, and Random Trees algorithms.

A. C5.0 Algorithm

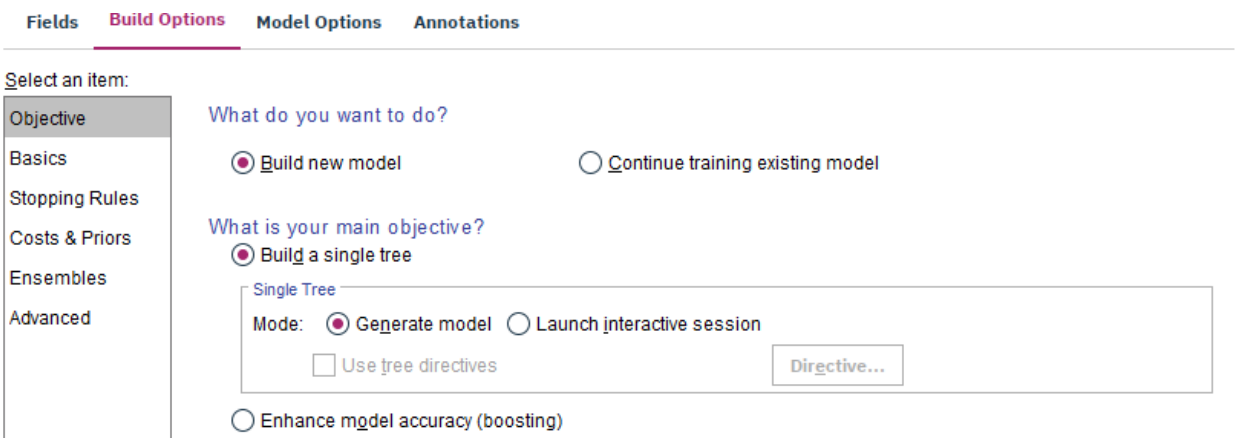
We used the C5.0 algorithm to mine the Boston housing dataset. Our target was set to the CAT.MEDV attribute while the rest of the attributes were used as inputs to the algorithm. We set the algorithm to output type to be as a decision tree and the mode was set to favor accuracy. Below are snapshots of all the settings used

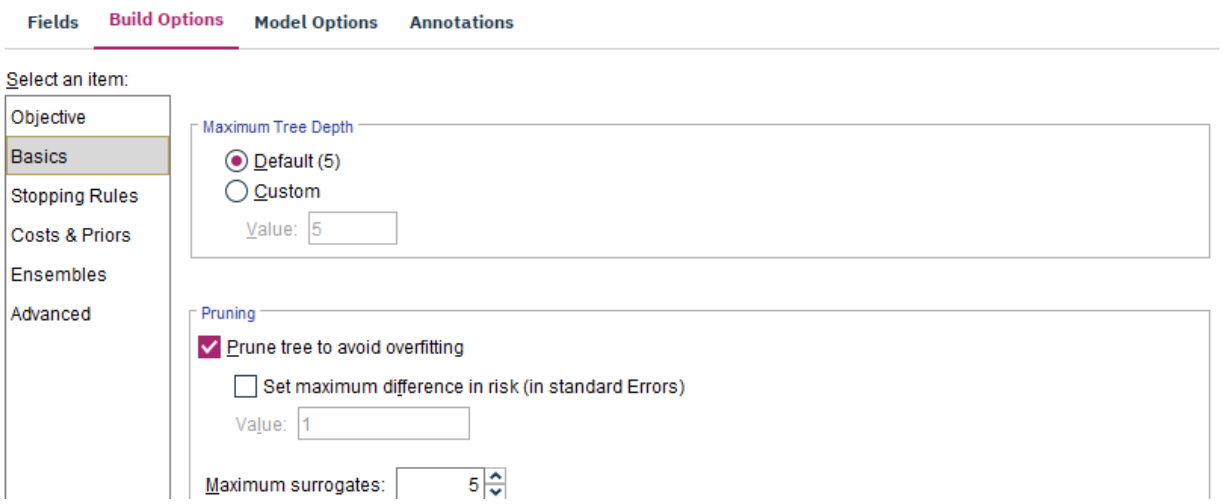




B. C&R Tree

The next algorithm we used in the SPSS Modeler was the C&R Tree algorithm. In a similar fashion to the C5.0 algorithm we set our target output to the CAT.MEDV attribute, while the remaining attributes were set to be inputs in the algorithm. In the build options we set the algorithm's objective to build a single tree with a maximum tree depth of 5 and allowed pruning to avoid overfitting. Below are snapshots of settings used.





Fields | Build Options | Model Options | Annotations

Select an item:

- Objective
- Basics
- Stopping Rules
- Costs & Priors
- Ensembles
- Advanced

(●) Use percentage

Minimum records in parent branch(%): 2.0

Minimum records in child branch(%): 1.0

( ) Use absolute value

Minimum records in parent branch: 100

Minimum records in child branch: 50

Fields | Build Options | Model Options | Annotations

Select an item:

- Objective
- Basics
- Stopping Rules
- Costs & Priors
- Ensembles
- Advanced

These settings determine the behavior ensembling that occurs when bagging, boosting, or very large datasets are requested in Objectives.
Options that do not apply are ignored.

Bagging and Very Large Datasets

Default combining rule for categorical targets: Voting

Default combining rule for continuous targets: Mean

Boosting and Bagging

Number of component models for boosting or bagging: 10

Fields | Build Options | Model Options | Annotations

Select an item:

- Objective
- Basics
- Stopping Rules
- Costs & Priors
- Ensembles
- Advanced

Minimum change in impurity: 0.0001

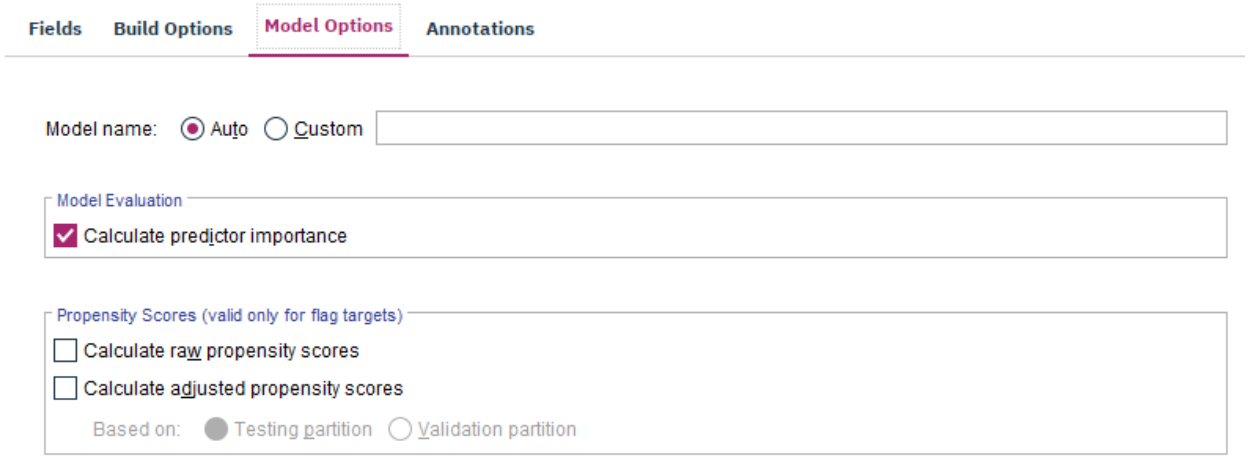
Impurity measure for categorical targets: Gini

Overfit prevention set(%): 30.0

[x] Replicate results

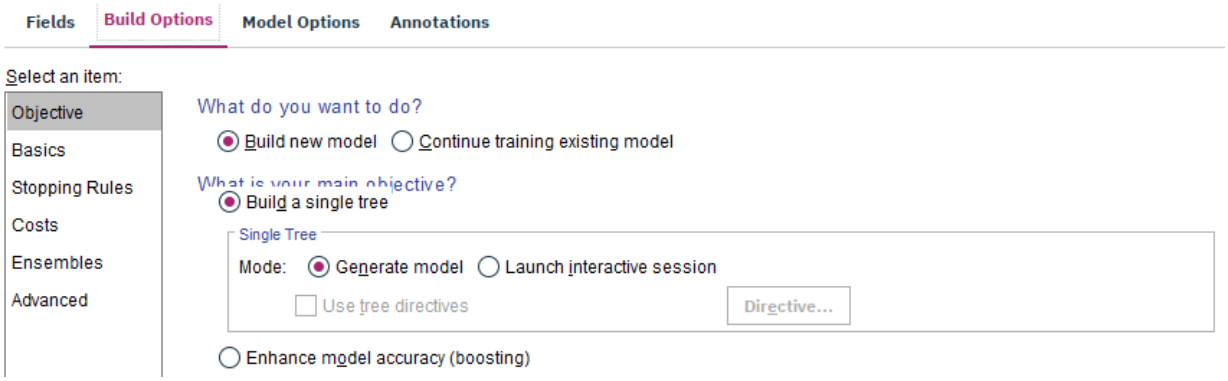
Generate

Random seed: 681644031



C. CHAID Algorithm

The next algorithm we used was the CHAID Algorithm. It was set to the same target attribute that the C&R tree algorithm was set to being CAT. MEDV. The inputs were also the same, consisting of all the other attributes excluding the CAT.MEDV attribute. The build objective for this algorithm was set to build a single tree with the CHAID growing algorithm. The Chi-Square for categorical targets was set to Pearson. The CHAID algorithm was run a second time with the Chi-Square for categorical targets set to likelihood ratio. Below are snapshots of settings used.



Fields | Build Options | Model Options | Annotations

Select an item:

- Objective
- Basics
- Stopping Rules
- Costs
- Ensembles
- Advanced

Tree growing algorithm: CHAID

Maximum Tree Depth

- (●) Default (5)
- ( ) Custom

Value: 5

Select an item:

- Objective
- Basics
- Stopping Rules
- Costs
- Ensembles
- Advanced

(●) Use percentage

Minimum records in parent branch(%): 2.0

Minimum records in child branch(%): 1.0

( ) Use absolute value

Minimum records in parent branch: 100

Minimum records in child branch: 50

Select an item:

- Objective
- Basics
- Stopping Rules
- Costs
- Ensembles
- Advanced

These settings determine the behavior ensembling that occurs when bagging, boosting, or very large datasets are requested in Objectives.
Options that do not apply are ignored.

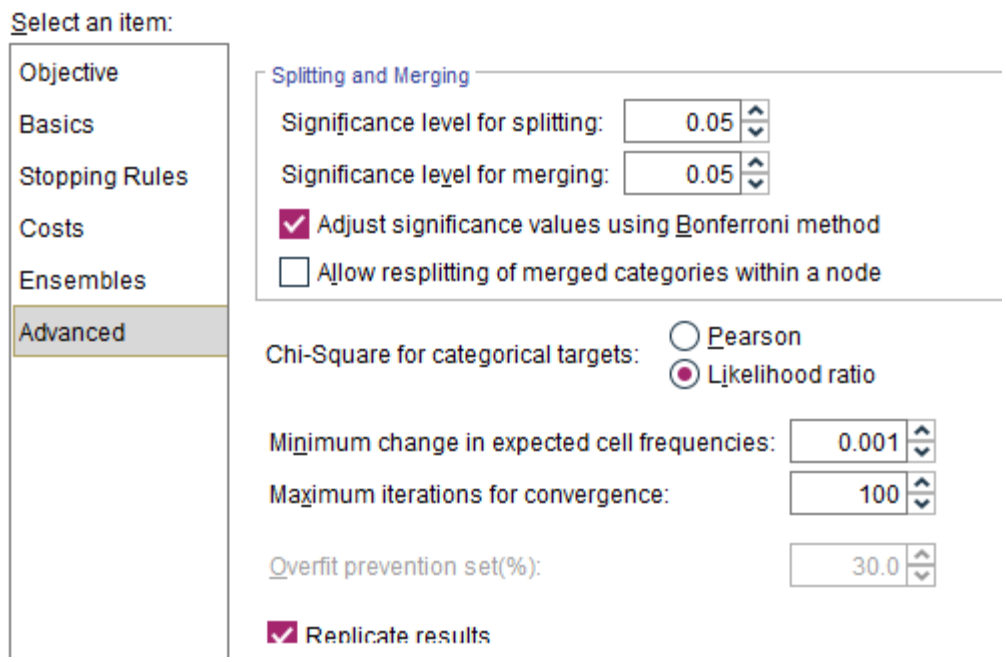
Bagging and Very Large Datasets

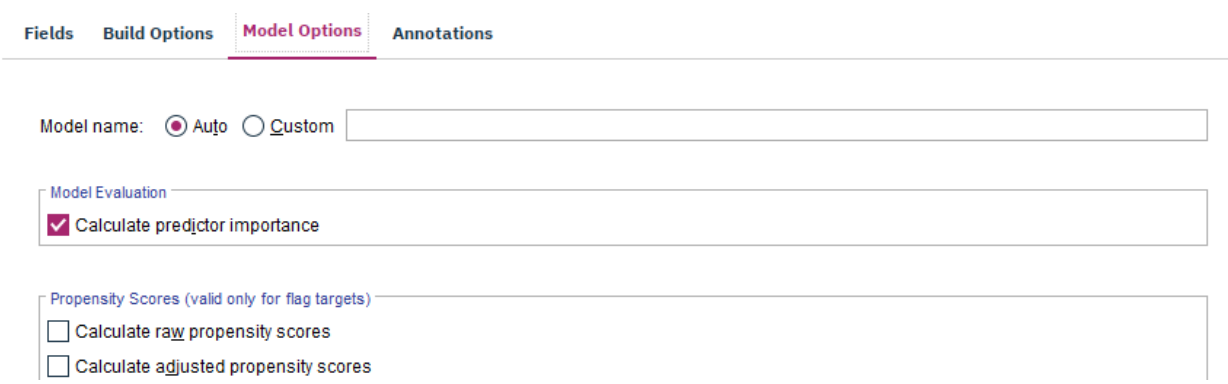
Default combining rule for categorical targets: Voting

Default combining rule for continuous targets: Mean

Boosting and Bagging

Number of component models for boosting or bagging: 10





D. QUEST Algorithm

The following algorithm we used was the QUEST algorithm. The target output was set to being CAT.MEDV while the remaining attributes were set to be the inputs. The model objective setting remains the same as previous models, being set to build a single tree. Below are snapshots of settings used.

Fields | Build Options | Model Options | Annotations

Select an item:

- Objective
- Basics
- Stopping Rules
- Costs & Priors
- Ensembles
- Advanced

What do you want to do?

(•) Build new model ( ) Continue training existing model

What is your main objective?

(•) Build a single tree

Single Tree

Mode: (•) Generate model ( ) Launch interactive session

[ ] Use tree directives Directive...

( ) Enhance model accuracy (boosting)

Select an item:

- Objective
- Basics
- Stopping Rules
- Costs & Priors
- Ensembles
- Advanced

Maximum Tree Depth

(•) Default (5)

( ) Custom

Value: 5

Pruning

[x] Prune tree to avoid overfitting

[ ] Set maximum difference in risk (in standard Errors)

Value: 1

Maximum surrogates: 5

Select an item:

- Objective
- Basics
- Stopping Rules
- Costs & Priors
- Ensembles
- Advanced

(•) Use percentage

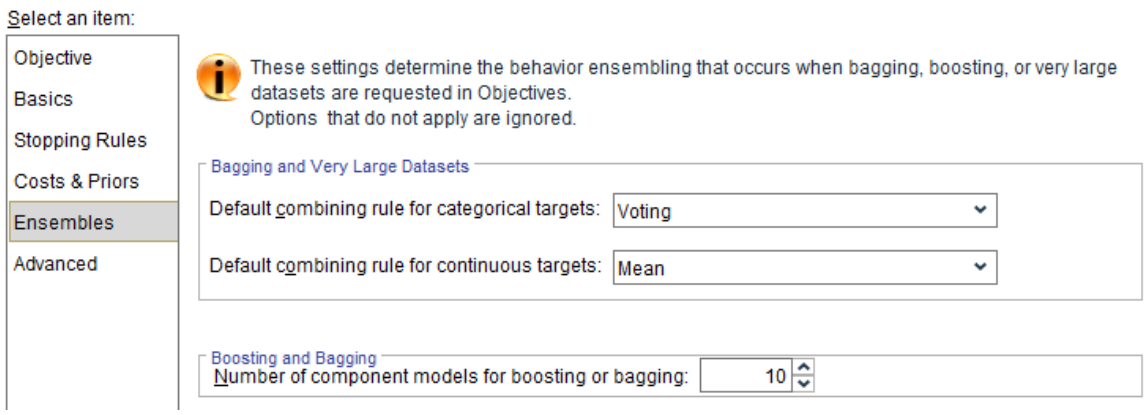
Minimum records in parent branch(%): 2.0

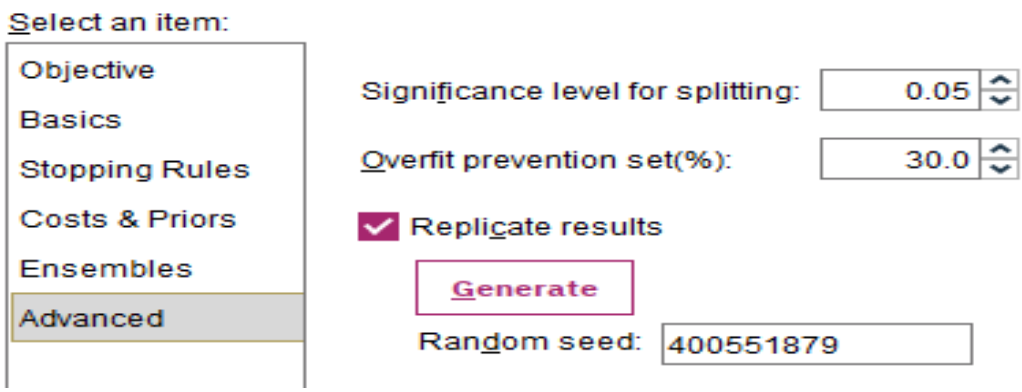
Minimum records in child branch(%): 1.0

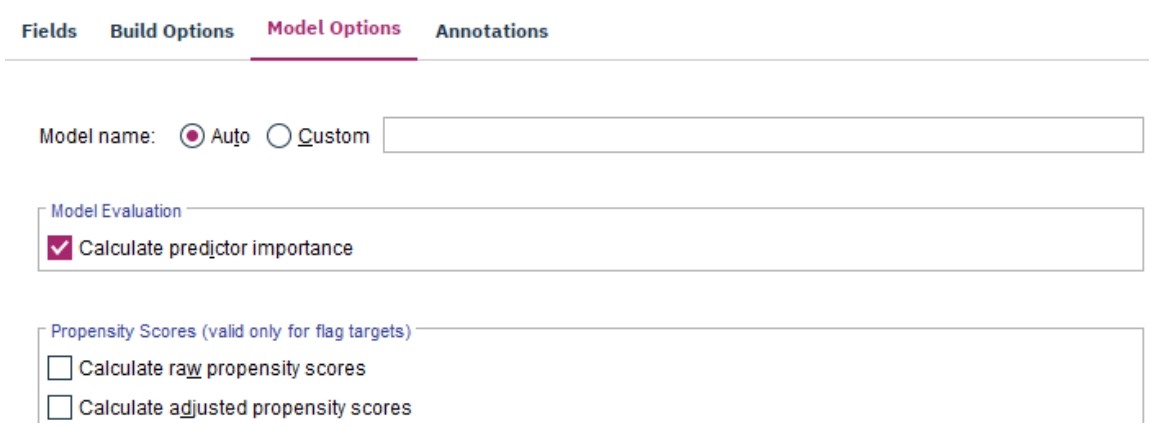
( ) Use absolute value

Minimum records in parent branch: 100

Minimum records in child branch: 50







E. Tree-AS Algorithm

The Tree-AS algorithm's target output was set to CAT.MEDV while the remaining attributes were once again set to be the inputs. Under the P value settings, the significance level for splitting and merging was set to 0.05. Under the Effect size settings, the Effect size threshold for continuous targets only was set to 0.04. The Chi-Square for categorical targets was set to Pearson. Below are snapshots of settings used.

Fields | Build Options | Model Options | Annotations

Select an item:

- Basics
- Growing Rules
- Stopping Rules
- Costs

Tree growing algorithm: CHAID

Maximum tree depth: 5

Binning

All continuous fields are automatically binned

Number of bins: 5

Select an item:

- Basics
- Growing Rules
- Stopping Rules
- Costs

Splitting and Merging

Record threshold for switching from p-values to effect sizes: 1000000

P-values settings:

Significance level for splitting: 0.05

Significance level for merging: 0.05

☑ Adjust significance values using Bonferroni method

Effect size settings:

Effect size threshold (continuous targets only): 0.04

Effect size threshold (categorical targets only): 0.2

☐ Allow resplitting of merged categories within a node

Significance level for grouping leaf nodes: 0.05

Chi-Square for categorical targets:

- (●) Pearson
- ( ) Likelihood ratio

Select an item:

- Basics
- Growing Rules
- Stopping Rules
- Costs

Minimum Records

(●) Use percentage

Minimum records in parent branch (%): 2

Minimum records in child branch (%): 1

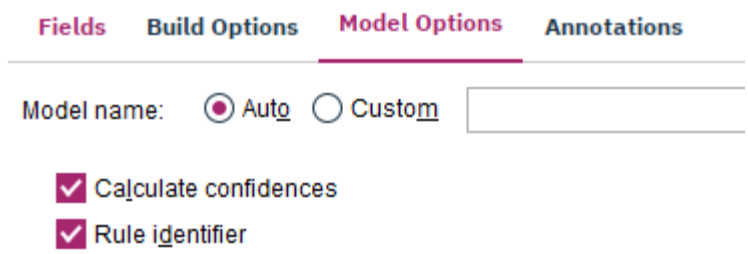
( ) Use absolute value

Minimum records in parent branch: 100

Minimum records in child branch: 50

Minimum change in expected cell frequencies: 0.001

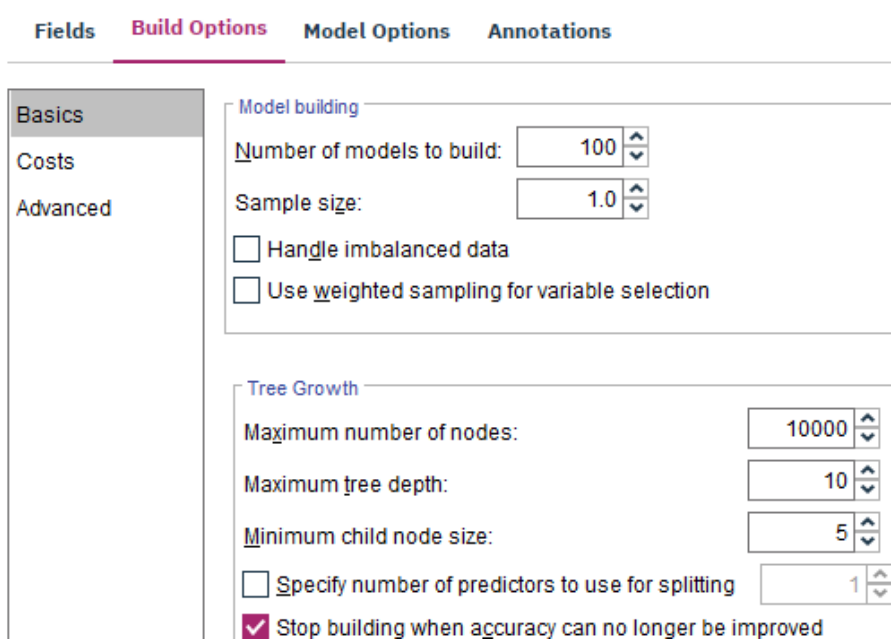
Maximum iterations for convergence: 100

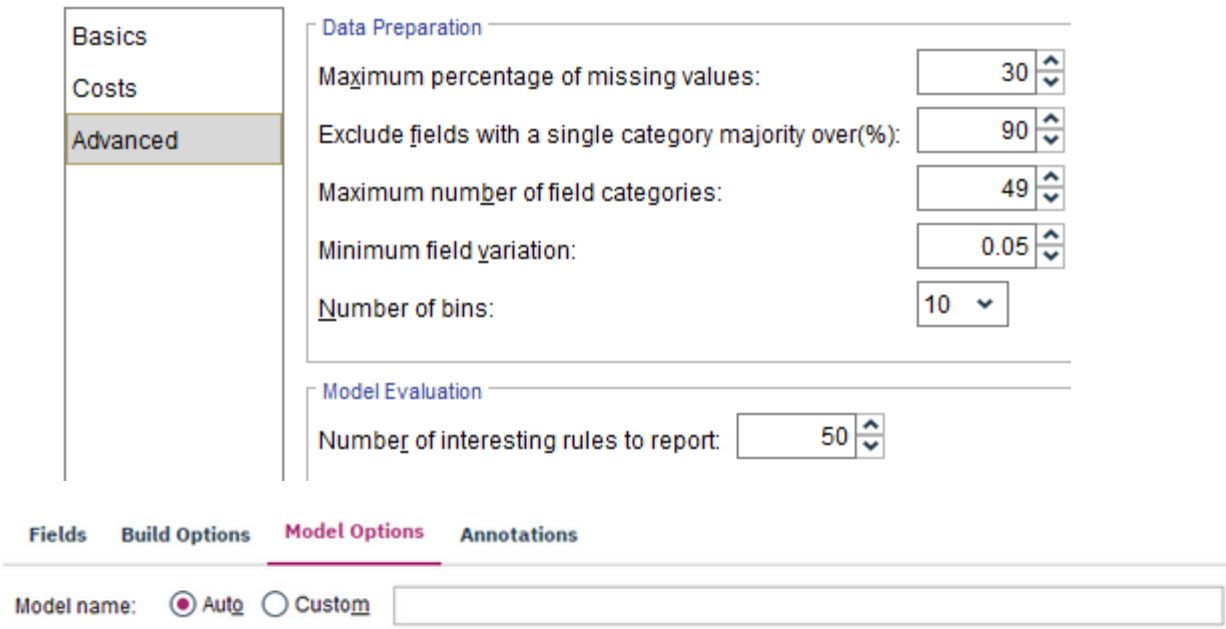


F. Random Tree Algorithm

In this node, the target attributes and input variables remain the same as in previous algorithms. The Random Tree algorithm is set to build 100 models with a sample size of 1. It has a maximum number of nodes set to 10000 with a max tree depth of 10 and a minimum child node size set to 5. The algorithm will stop building when accuracy can no longer be improved.

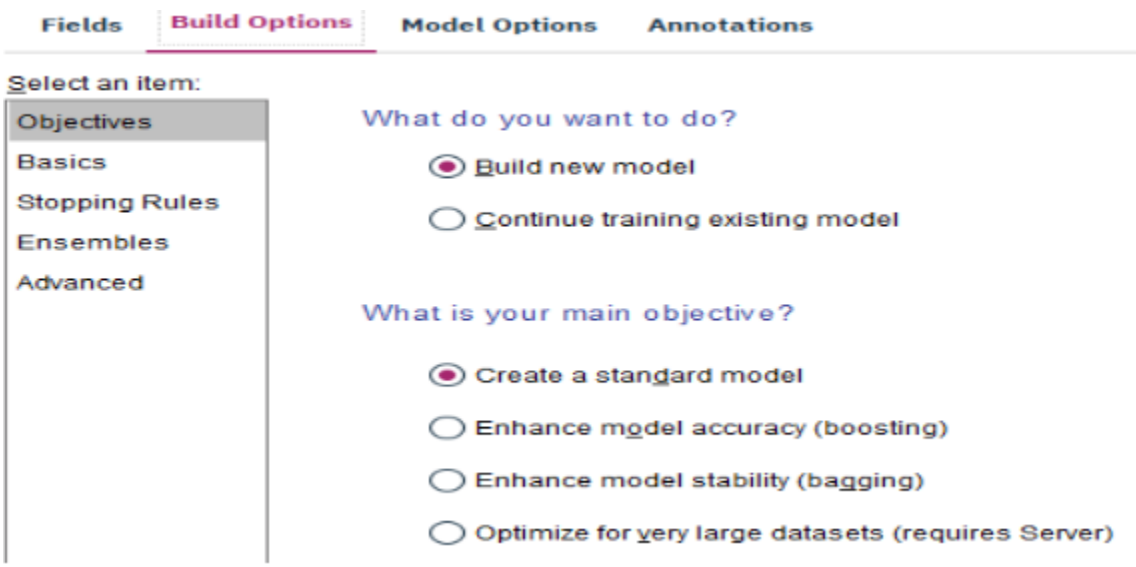
In the data preparation settings, the maximum percentage of missing values is set to 30%. The algorithm will exclude fields with a single category majority over 90%. There is a maximum of 49 field categories and the minimum field variation is set to 0.05. Below are snapshots of settings used.





G. Neural Network

In this node, the target attribute settings remain the same for the input attributes and the target attribute. For this Neural network model, we used the Multilayer Perceptron (MLP) option. The hidden layers were set to automatically compute the number of units. Below are snapshots of settings used.



Fields | Build Options | Model Options | Annotations

Select an item:

- Objectives
- Basics
- Stopping Rules
- Ensembles
- Advanced

Neural network model: Multilayer Perceptron (MLP)

Hidden Layers

- (●) Automatically compute number of units
- ( ) Customize number of units

Hidden layer 1: 1

Hidden layer 2: 0

Fields | Build Options | Model Options | Annotations

Select an item:

- Objectives
- Basics
- Stopping Rules
- Ensembles
- Advanced

Stopping rules are available for the Multilayer Perceptron model and apply to training the standard model (or component models if ensembles are in effect). Other stopping rules apply.

- [x] Use maximum training time (per component model)

Minutes: 15

- [ ] Customize number of maximum training cycles

Maximum number of cycles: 250

- [ ] Use minimum accuracy

Accuracy (%): 90.0

The maximum training time does not include the time for the calculation of

Select an item:

- Objectives
- Basics
- Stopping Rules
- Ensembles
- Advanced

These settings determine the behavior of ensembling that occurs when boosting, bagging, or very large datasets are requested in Objectives. Options that do not apply are ignored.

Bagging and Very Large Datasets

Default combining rule for categorical targets: Voting

Default combining rule for continuous targets: Mean

Boosting and Bagging

Number of component models for boosting and/or bagging: 10

Select an item:

- Objectives
- Basics
- Stopping Rules
- Ensembles
- Advanced

Neural Network modeling internally separates records into a model building set and an overfit prevention set. Specify a percentage of records for the overfit prevention set.

Overfit prevention set (%): 30.0

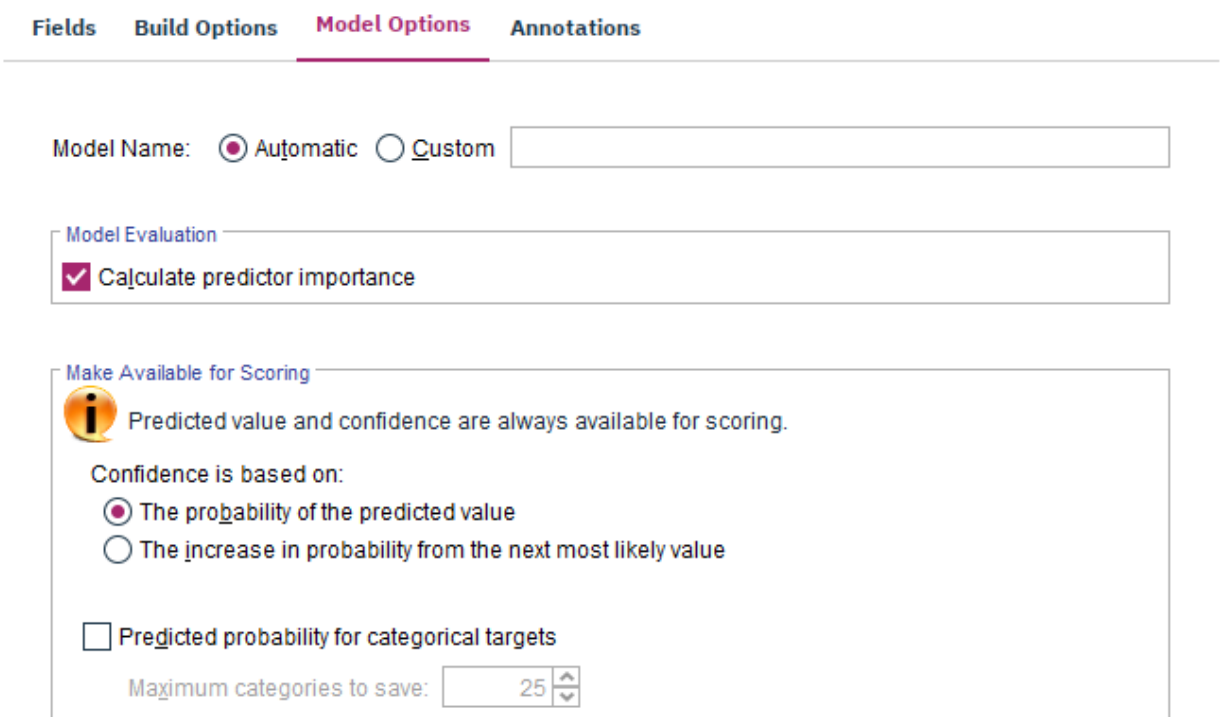
☑ Replicate Results

Generate

Random seed: 229176228

Missing values in predictors:

- (●) Delete listwise
- ( ) Impute missing values



H. Accuracy of nodes

This table lists the Training Accuracy, Testing Accuracy, and overfit for each algorithm.

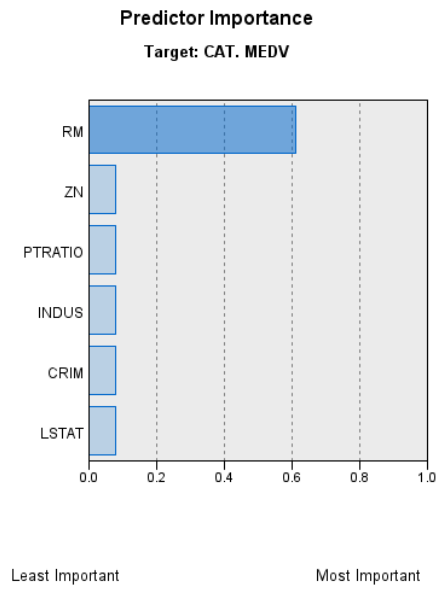
Model	Accuracy Training	Accuracy Testing	Overfit
C5.0	99.37%	90.48%	8.89%
C&R Tree	94.3%	90.48%	3.82%
CHAID-Pearson	98.1%	85.71%	12.39%
CHAID-Likelihood	96.84%	90.48%	6.36%
QUEST	93.67%	90.48%	3.19%
Tree-AS	96.84%	90.48%	6.36%
Random Trees	98.73%	95.24%	3.49%

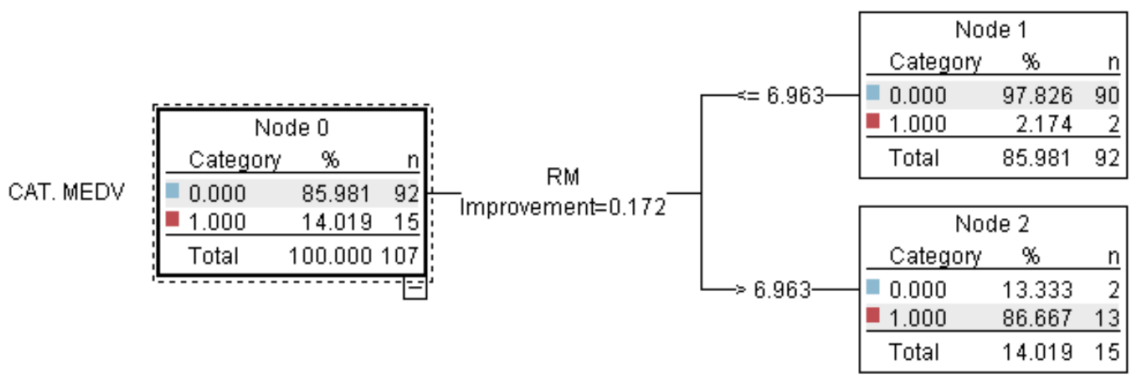
Neural Network	95.57%	92.86%	2.71%

IV. Results

After looking at the accuracy tables, we can see that some of these models overfit the data, and other models do not overfit it as much. We would want to use the data sets that have the least overfitting, as these will have fewer errors, to predict the median value of owner-occupied homes. This would mean that we would use the C&R Tree, Quest, Random Trees, and Neural Network models as they are the only ones that fall below 4% overfitting.

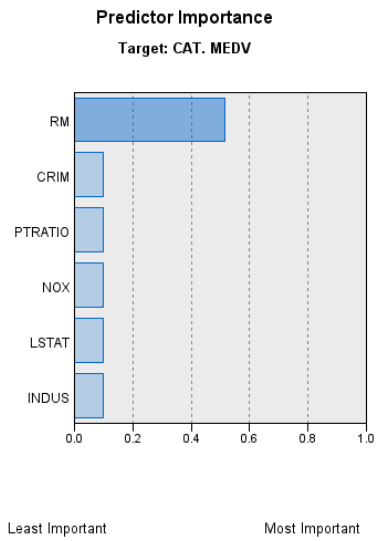
If we look at the C&R Tree model, we can see that the average number of rooms per dwelling is the input that has the most importance when predicting the median value of owner-occupied homes. The RM predictor importance was equal to 0.61, while the other inputs were equal to 0.08. This model also outputs a decision tree using this input for the predictor. This decision tree states that if the average number of rooms per owner-occupied home is greater than or equal to 6.963, then the home is worth below $300,000, and if it is less than or equal to 6,963, then the home is worth more than $300,000.

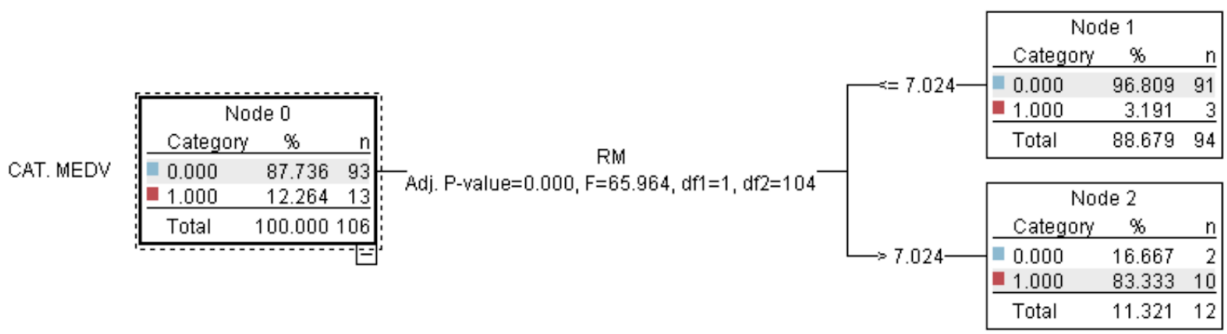




If we look at the next model, Quest, we can see that this model had the same input, RM, as the leading predictor at 0.52 predictor importance. All of the other inputs had a predictor importance of 0.1. The decision tree that the model output was also similar; however, this time, the determining RM value was 7.024. If the RM was greater than or equal to this, the house was less than $300,000, similar to the C&R Tree model, and if the RM was less than this value, it was greater than $300,000. When comparing the accuracy of these two models, the C&R Tree is

slightly more overfitting than the Quest, but its training data is more accurate; this is displayed in the accuracy table and here.





Now, if we look at the Random Trees model, we can see that it uses all twelve inputs and has an estimated accuracy of 93%. The predictor importance graph is different from the last ones, with RM still being the top predictor; this time, the predictability importance is at 1.0, and the other variables have a range of importance values. Instead of a decision tree, this graph outputs a table filled with the top decision rules for our given input. This table includes the Rule Accuracy, Ensemble Accuracy, Interestingness Index, and most frequent category. The rule that was the best in this model was the rule that "if the proportion of residential land zone for lots over 25,000 sq. ft. was less than or equal to 20 and the percentage of the lower status of the

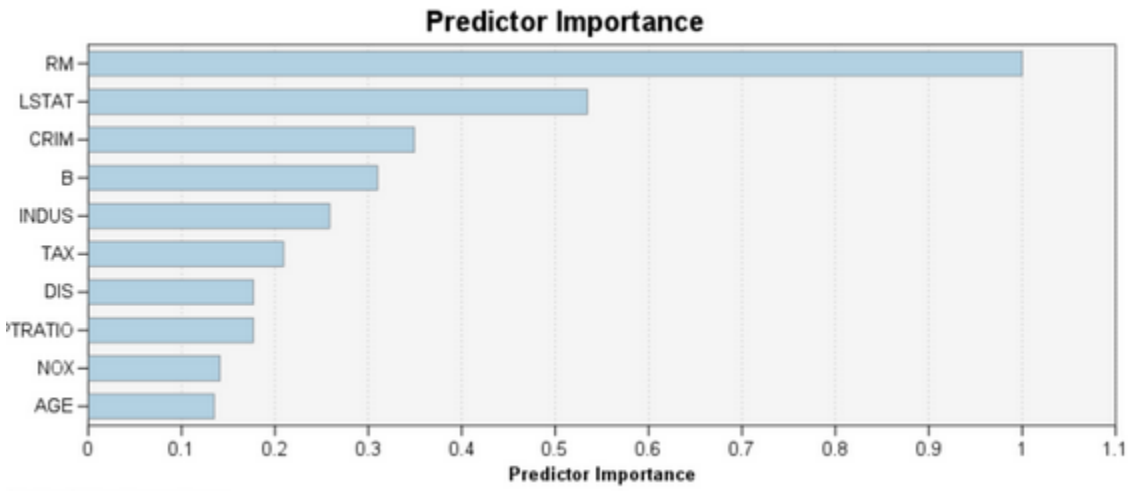
population was less than or equal to 6.07 and per capita crime rates by town are in between 0.03659 - 9.72418 and average number of rooms per dwelling is greater than 6.538, then the median value of the home is greater than 300,000. This seems like the best model for the data, as so far, it is the only one that utilizes more than one input in its rules. However, using more inputs can lead to more classification errors.

Model Information

Target Field	CAT. MEDV
Model Building Method	Random Trees Classification
Number of Predictors Input	12
Estimated Model Accuracy	0.930
Estimated Misclassification Rate	0.070

Records Summary

Records	Number	Percent
Included	158	100.00
Excluded	0	0.00
Total	158	100.00



The top 10 inputs are shown.

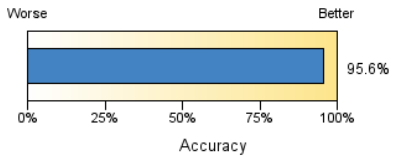
Top Decision Rules for 'CAT. MEDV'

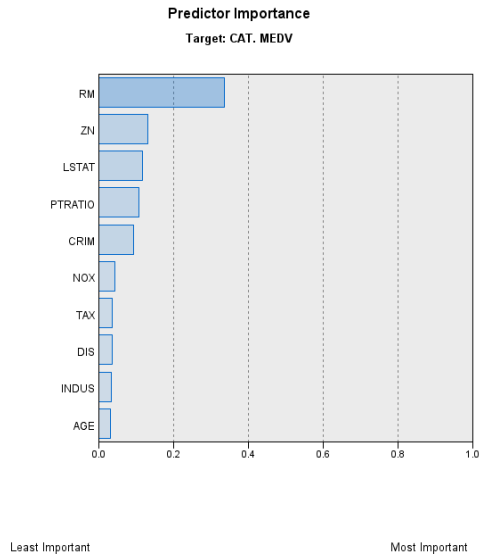
Decision Rule	Most Frequent Category	Rule Accuracy	Ensemble Accuracy	Interestingness Index
(ZN <= 20.0) & (LSTAT <= 6.07) & (CRIM > 0.03659) & (CRIM <= 9.72418) & (RM > 6.538)	1.0	1.000	1.000	1.000
(RM <= 6.538)	0.0	1.000	1.000	1.000
(LSTAT > 6.07) & (CRIM > 0.2909) & (INDUS <= 18.1) & (INDUS > 5.13)	0.0	1.000	1.000	1.000
(CRIM <= 0.55007) & (INDUS > 18.1) & (INDUS > 5.13)	0.0	1.000	1.000	1.000
(RM <= 6.538) & (INDUS <= 5.13)	0.0	1.000	1.000	1.000

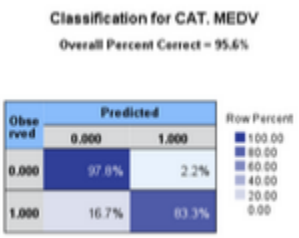
The final model, Neural Networks, has four outputs. The first output is a model summary that gives information about the model. The model's accuracy is 95.6%, which makes sense because this is training data, and that is the same value as our accuracy tables training data. The next output is the predictor importance of the inputs. This model, along with the others, has the average number of rooms per dwelling as the most important predictor. For this model, the predictor importance is 0.34, which is the lowest of the models used. The other inputs are a range of data similar to the Random Trees model. The next output is a simple confusion matrix for the target output. This confusion matrix is good because you would rather underpredict the value of a house than overpredict. This is because if you overpredict the value, you could be overcharging or overspending on a house that is not worth that price. Also, from this, we can calculate the model's lift, which is 1.676. This gives us a better idea as to how this model compares with the others. The final output of this model is a Multi-layer Neural Network. When changing the style of the network from effects to coefficients, we can see the weight of each node and the impact of the weight, positive or negative. This gives us a great understanding of our inputs and the relation that they have with the median value of owner-occupied homes.

Model Summary

Target	CAT. MEDV
Model	Multilayer Perceptron
Stopping Rule Used	Error cannot be further decreased
Hidden Layer 1 Neurons	6





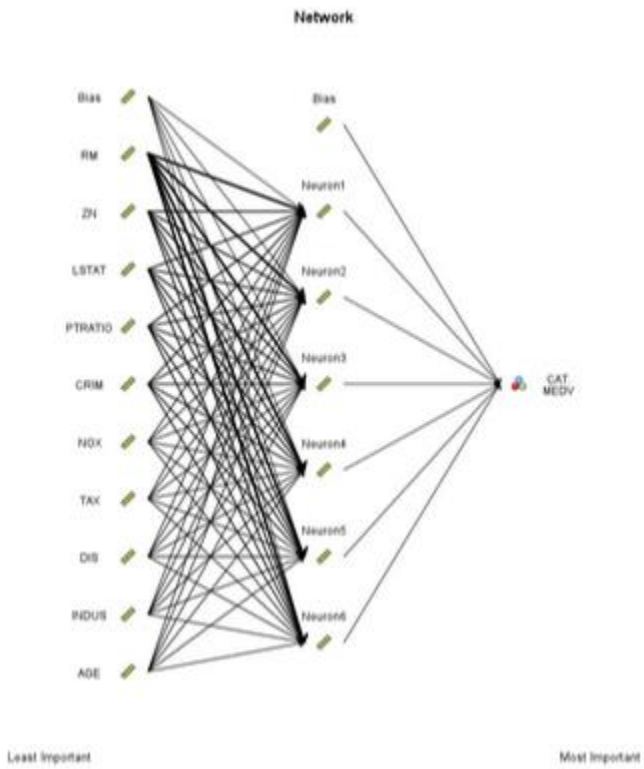


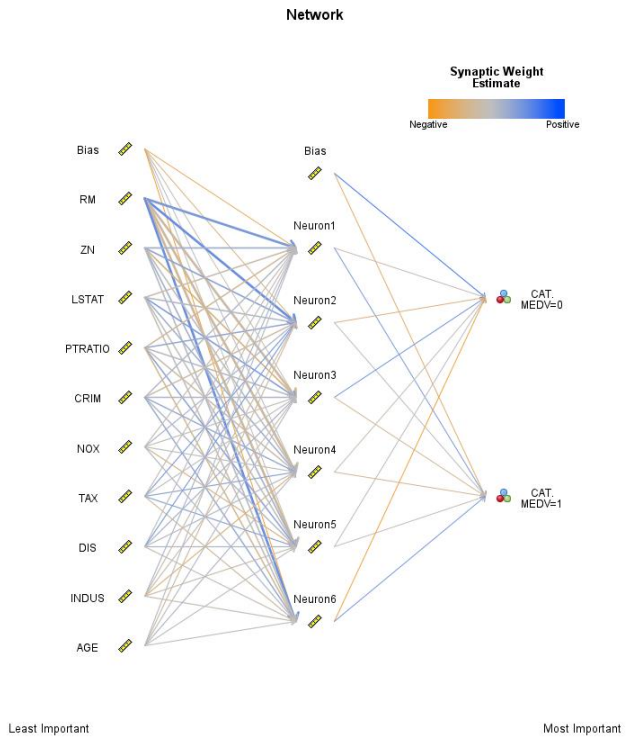
Classification for CAT. MEDV

Overall Percent Correct = 95.6%

Observed	Predicted 0.000	Predicted 1.000
0.000	97.8%	2.2%
1.000	16.7%	83.3%

Row Percent: 100.00, 80.00, 60.00, 40.00, 20.00, 0.00





V. Recommendations

With our findings, the US Census Service would be better off using a mixture of the Random Trees model and the Neural Networks model to create a plan for the future. Even though the Random Trees model is not the most accurate, it is in the top three, and on top of that, it uses rules that the other models do not use. This can allow them to find factors that affect the median value of owner-occupied homes that other models would not. Additionally, they should use Neural Networks to see and measure the impact that a specific input has on the target. With all of the graphs determining that the RM value is the most important for the CAT MEDV, they should use that information along with some of the other stronger inputs to determine the median value of owner-occupied homes more accurately.

CONTRIUTION REPORT – Caden 100%, Colton 100%, Trevor 100%, Jarryn 100%